

HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Distributed Computing

Michael Kosyakov
Associate Professor

Denis Tarakanov
Assistant Lecturer

ifmo.distributedclass.bot@gmail.com



Logical clocks

Logical clocks

Physical timestamps are useful for many things, but may be **inconsistent with causality**: effect may have an older timestamp than the cause

Physical clock count number of **seconds elapsed**

Logical clock count number of **events occurred**

-- Martin Kleppmann

Causality
Why is it important?



- Last Writer Wins with physical clocks: Alice and Bob edit shared document replicated at their respective sites, but Bob's wall-clock is one hour ahead of Alice's
- Alice updates the document at 11:00 (12:00 at Bob)
  - Alice's update casually dominates version stored at Bob
- After the sync, Bob updates the document at 12:15
  - After the sync, the document timestamped with 12:15 is not writeable at Alice until her clock advances to 12:15
- Even if we allow Alice to overwrite Bob's changes at 11:30, her updates will be lost on sync since 12:15 > 11:30
  - Causally new data gets older timestamp than old data

Recap

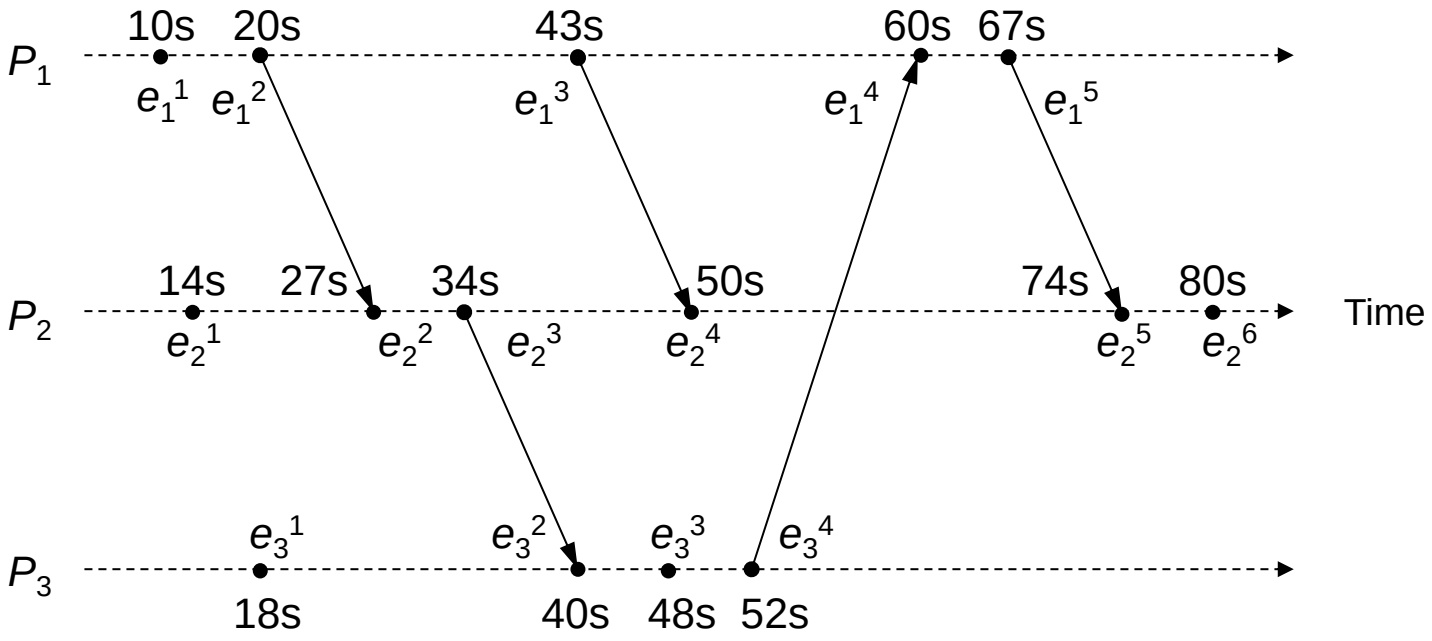
- There is no common global clock in distributed systems, hence, we had to define “happens before” relation without relying on the concept of a real time
- We showed that traditional notion of time as a total order on the events of an execution is not suitable for distributed executions since “happens before” is a partial order and the processes can’t determine the actual order of events in the absence of accurately synchronized physical clocks
- If the system had a shared clock (or, equivalently, precisely synchronized clocks) then timestamping the event would be sufficient to determine the order, but unfortunately we do not have such a thing …

Logical clock

- We will develop a mechanism that gives a total order that *could have* happened instead of the total order that did happen, i.e. we'll get one of the equivalent executions
- The purpose of logical clock is only to order events and not to provide any other property associated with clocks
  - No chance to determine the time elapsed between two events
- Unlike physical time, logical time does not flow by itself
  - No chance to do nothing and wait for a certain moment of logical time in the future
- Logical clock ticks only when events occur in distributed system, hence logical time is discrete

Physical clock

If we had a global real time clock that would assign a timestamp to each event…



We would observe that the total order is consistent with the causal order,
i.e. real time preserves the causal order
Note that while 48s < 50s corresponding events $e_3^3 \parallel e_2^4$ are concurrent in causal order

Logical clock

- What does it mean when we say that “something happened at 9:30”? What is time and what is clock?
- Logical clock Θ is a function that maps an event *e* in a distributed computation to an element in the time domain *T*, denoted as Θ(*e*) and called the timestamp / “logical time” of *e*
- Being consistent with causality if often captured by a *clock consistency condition*:

$$\forall e_i, e_j' \in E: e_i \rightarrow e_j' \Rightarrow \Theta(e_i) < \Theta(e_j')$$

- Under these assumptions, we will be able to show that the logical-time assignment "looks like" a real-time assignment to the processes

Implementing logical clocks



- In distributed systems each process P_i can only control its own clock Θ_i, regardless of the clocks of other processes
- The logical timestamp of the event e_i that occurs in the process P_i will be determined by the clock readings of this process at the time of event occurrence, i.e. $\Theta(e_i) = \Theta_i(e_i)$ for all events e_i of the process P_i
- The value of Θ_i changes *between* events of the process P_i, i.e. changing Θ_i does not itself constitute an event
- But how do we satisfy clock consistency condition if clocks Θ_i are managed independently by each process P_i?

Implementing logical clocks



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- Clock consistency condition is satisfied if the following two conditions hold (follows from the definition of causal order)
  - **C1**. If e_i and e_i' are events in process P_i, and e_i comes before e_i', then $\Theta_i(e_i) < \Theta_i(e_i')$
  - **C2**. If e_i is the sending of a message by process P_i and e_j' is the receipt of that message by process P_j, then $\Theta_i(e_i) < \Theta_j(e_j')$
- Implementation of logical clocks requires defining
  - 1) data structures local to every process to represent logical time, and
  - 2) set of rules to advance the clock, i.e. how to update the data structures to ensure the consistency condition

Implementing logical clocks



- Data structures for representation of logical time usually provide following two capabilities and consist of two parts:
- A local logical clock, for process P_i to measure its own progress
- A logical global clock, that is a representation of process P_i's local view of the logical global time and global progress
- Logical global clock allows every process to assign consistent timestamps to its local events

Implementing logical clocks



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- The clock advancing protocol should satisfy clock consistency condition and is defined by two rules:
- **R1**. This rule defines how P_i updates its own local logical clock when it executes an event (send, receive, or internal)
- **R2**. This rule defines how P_i updates its logical global clock to update its view of the global time and global progress (it dictates what information about the logical time is piggybacked in a message and how this information is used by the receiving process to update its view of global time)
- R1 and R2 should satisfy C1 and C2 conditions listed above

Scalar time

- Lamport proposed the scalar time representation for totally ordering events in a distributed system
- In this representation, the time domain T is the set of non-negative integers, so “global time” is just monotonically increasing number
- The logical local clock of a process P_i and its local view of the global time are represented by single integer variable L_i, and let $L(e_i)$ denote timestamp of event e_i
- The clock advancing protocol ensures that Lamport clock guarantees a correct system of logical clocks, i.e.

$$e_i \rightarrow e_j' \Rightarrow L(e_i) < L(e_j')$$

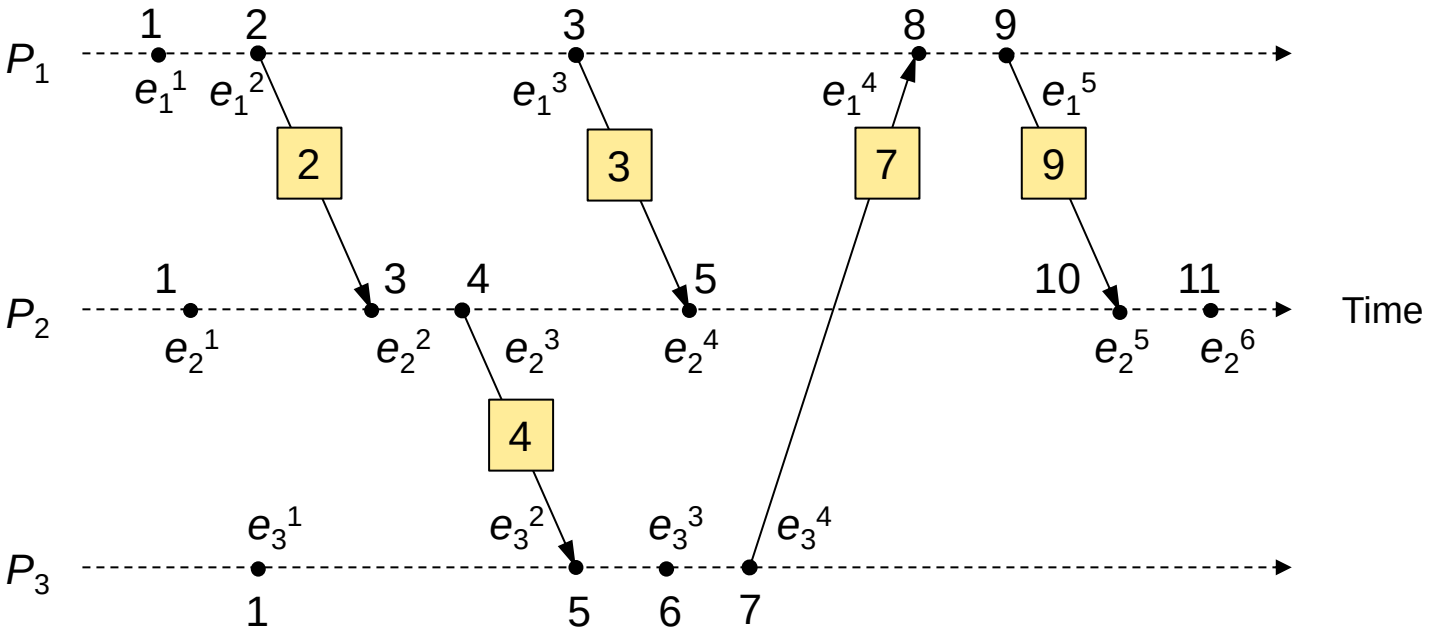
Lamport clock advancing protocol



- **R1**. Each process P_i increments L_i between any two successive events (send, receive, internal):
  - $L_i = L_i + d$, $d > 0$ (typically $d = 1$, L_i initialized with 0)
- **R2**. Each message piggybacks clock value L_{msg} of its sender at sending time, so that receiving process P_j could update its view of global time. When P_j receives a message with timestamp L_{msg}, it executes the actions:
  - $L_j = \max(L_j, L_{msg})$
  - Executes R1
  - Delivers the message
- Clearly, R1 and R2 satisfy both C1 and C2 clock conditions

Example of distributed execution

Evolution of scalar time in distributed execution



Note how P_3 advances its clock when receiving a message from P_2 with P_2's view of global time. Also, note that equal timestamps can only be assigned to concurrent events, i.e. $L(e_i) = L(e_j') \Rightarrow e_i \parallel e_j'$

Lamport clock

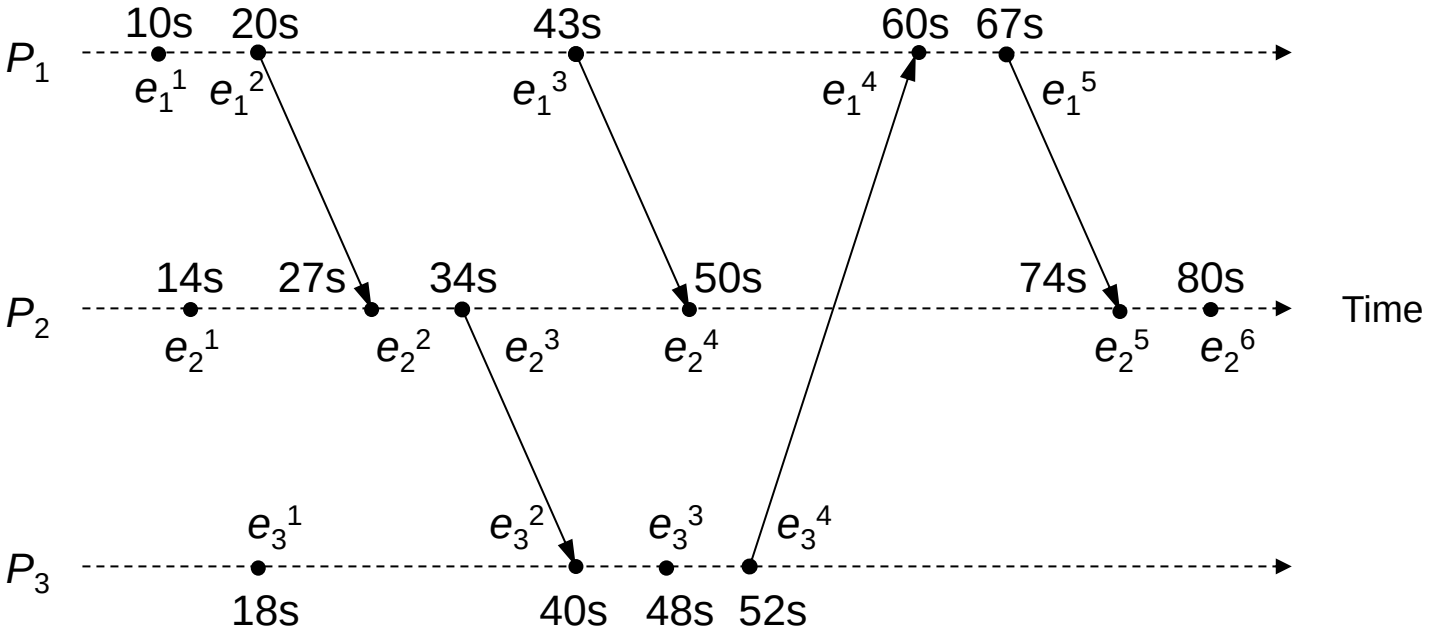
- Lamport clock satisfies clock consistency condition, hence, we can use Lamport timestamps to totally order all system events preserving causal order
- Two or more events at different processes having an identical timestamp are concurrent in causal order and can be ordered arbitrary without violating the causality relation
- Typically, process identifiers are linearly ordered and can be used to break ties, i.e.

$$e_i < e_j' \Leftrightarrow (L(e_i) < L(e_j')) \vee ((L(e_i) = L(e_j')) \wedge (i < j))$$

- Timestamp of an event is denoted by a tuple $(L(e_i), i)$

Physical clock

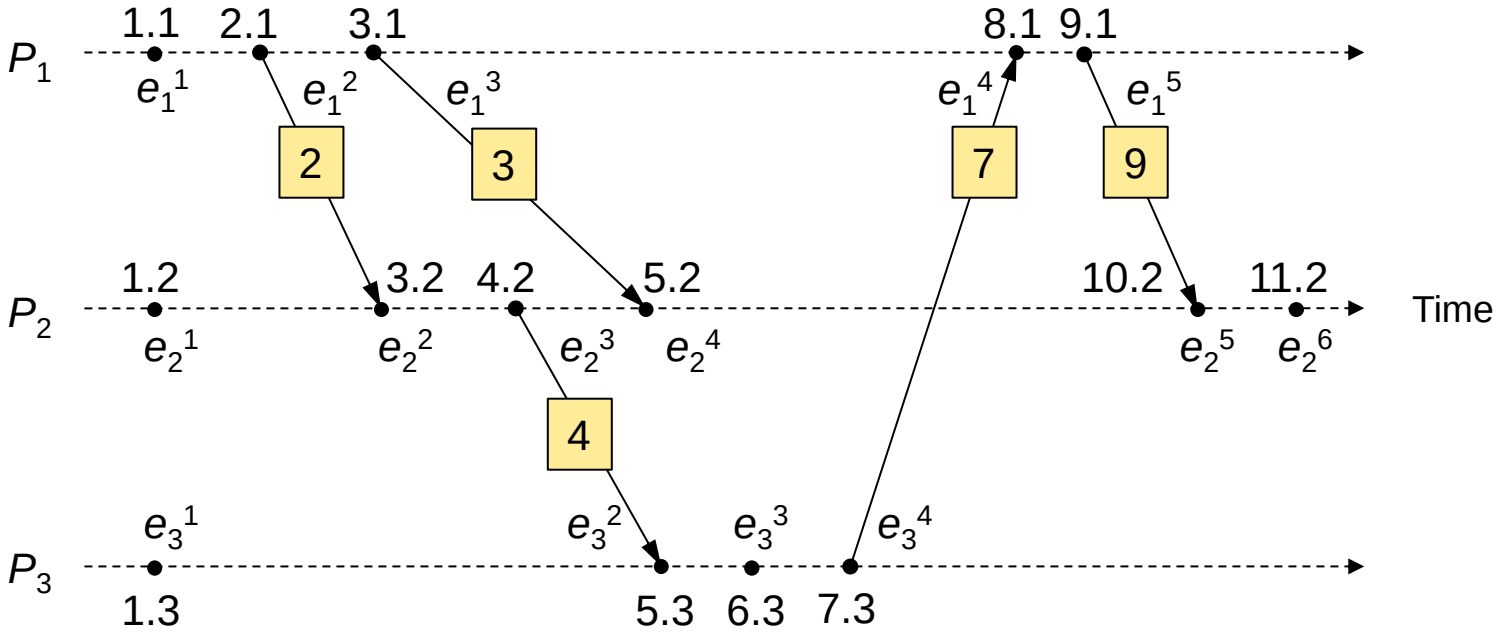
So while an ideal global observer possessing real time clock would observe the following execution…



The total order based on real time is consistent with the causal order …

Lamport clock

… we get another equivalent execution, where events occur in the order of their Lamport time



… and so does the total order based on Lamport time

Lamport clock

- We know that processes can't distinguish between two equivalent executions, so any Lamport-time assignment "looks like" a real-time assignment to every process in the distributed system
  - In this sense Lamport time behaves like real time
- Total order defined above imposes static priorities for the processes based on their identifiers
- We could define "more fair" tie-breaking mechanism as a function of logical time $L(e_i)$
- E.g. if $(L(e_i) = L(e_j')) \wedge (i < j)$ then we could set $e_i < e_j'$ if the value $L(e_i)$ mod N falls into interval $(i, j]$, and $e_j' < e_i$ otherwise

Lamport clock

- A total order is generally used to ensure liveness properties in distributed algorithms: requests are timestamped and served according to the total order based on these timestamps
  - It doesn't necessarily work for physical timestamps
- Please note that Lamport clock is not *strongly consistent*:

$$L(e_i) < L(e_j') \text{ doesn't imply that } e_i \rightarrow e_j'$$

- If $L(e_i) < L(e_j')$ we can only say, that $e_j' \not\rightarrow e_i$, i.e. either $e_i \rightarrow e_j'$ or $e_i \parallel e_j'$, so Lamport clock doesn't express concurrency
  - Mathematically this can be explained by the fact that the function $L(e)$ maps a partially ordered set of distributed events to a linearly ordered set of non-negative integers

Lamport clock: Banking system



- Lets reconsider the problem of counting the total amount of money in a banking system in which money is transferred between processes via messages
- We assume that each process sends infinitely many messages to each of its neighbors
  - It is always possible to add dummy messages containing $0
- Message ordering is preserved by a channel, i.e. each channel acts as a FIFO message queue
- Each process should decide on a local balance, in such a way that the total of all the balances is the correct amount of money in the system

Lamport clock: Banking system



- In order to avoid the difficulties related to using physical time we will calculate the amount of money at the predetermined logical time $t \in Z_{\geq 0}$, assumed to be known to all processes
- For each process P_i we will determine the value of the local balance after all events with logical times less than or equal to t and before all events with logical times greater than t
  - P_i should record its local state right before executing the first event e_i such that $L(e_i) > t$
  - There must be such an event e_i, because P_i performs infinitely many events and there are only finitely many events with logical time less than or equal to t

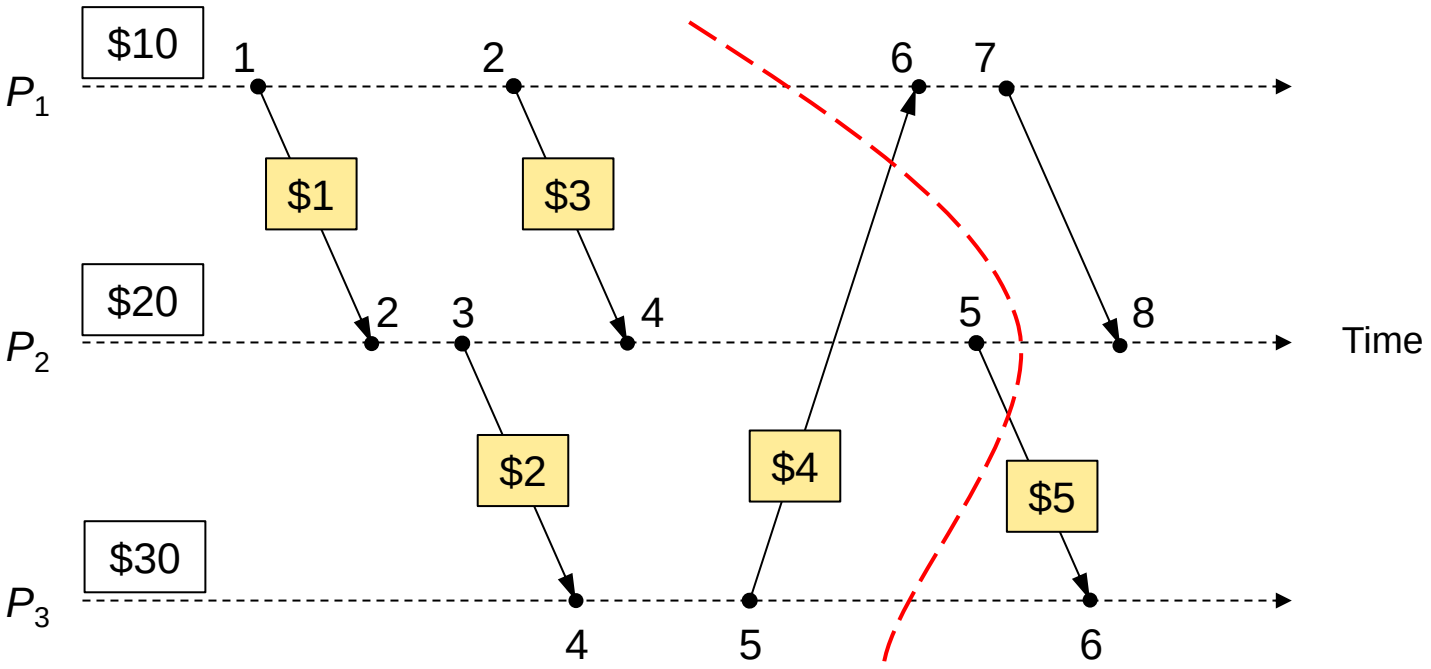
Lamport clock: Banking system



- In order to avoid the difficulties related to using physical time we will calculate the amount of money at the predetermined logical time $t \in Z_{\geq 0}$, assumed to be known to all processes
- For each channel C_{ji} we will determine the amount of money in all the messages sent at logical times less than or equal to t but received at logical times strictly greater than t
  - Starting with the first event e_i such that $L(e_i) > t$ (i.e. the one at which P_i records its local state) P_i should record messages coming in on the channel C_{ji} and having $L_{msg} \leq t$
  - When a message with $L_{msg} > t$ arrives on the channel C_{ji}, P_i returns the sum of the amounts of money in the recorded messages

Lamport clock: Banking system

Example of distributed execution with a logical-time assignment (t = 5)



Dotted line indicates time point t = 5 that intersects the process time lines

Lamport clock: Banking system

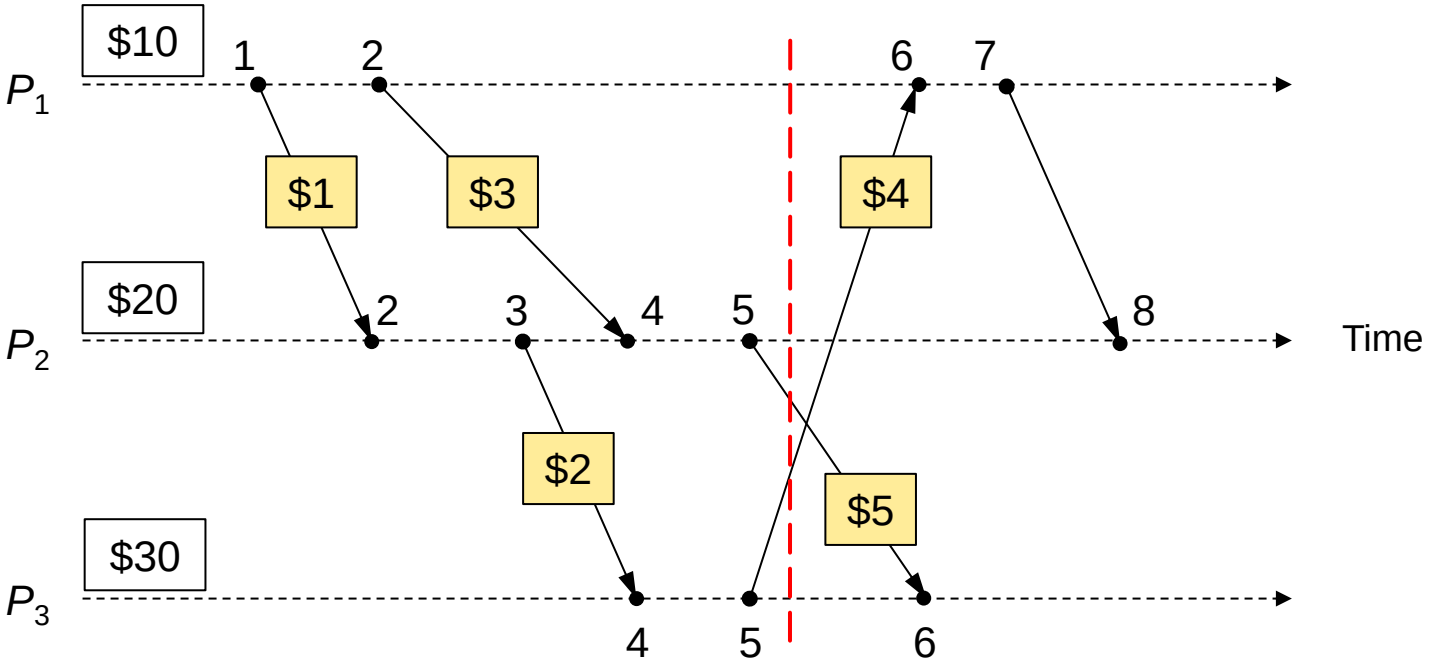


Execution of banking algorithm (t = 5)

- P_1: \$10 – \$1 – \$3 = \$6
- P_2: \$20 + \$1 – \$2 + \$3 – \$5 = \$17
- P_3: \$30 + \$2 – \$4 = \$28
- C_{31}: \$4
- C_{23}: \$5
- Total: \$6 + \$17 + \$28 + \$4 + \$5 = \$60

Lamport clock: Banking system

Equivalent execution in which the events appear in logical-time order ($t = 5$)



Dotted line gives an instantaneous global snapshot of the system during this execution (which is indistinguishable from the original one to all the processes), and must certainly yield the correct total amount of money in the banking system